

Loops in Programming

A Complete Beginner's Guide

Loops are one of the most fundamental concepts in programming. They allow you to execute a block of code repeatedly — saving time, reducing errors, and making your programs powerful and efficient.

Loop Type	Use When
for loop	You know how many times to repeat
while loop	You repeat until a condition is false
do-while loop	You need to run at least once
for-each loop	You iterate over a collection/array

1. The for Loop

A **for loop** repeats a block of code a specific number of times. It uses a counter variable that changes on each iteration.

Syntax (C / Java / JavaScript):

```
for (initialization; condition; update) {  
  // code to repeat  
}
```

Example — Print numbers 1 to 5:

```
for (int i = 1; i <= 5; i++) {  
  System.out.println("Number: " + i);  
}  
  
// Output:  
// Number: 1  
// Number: 2 ... Number: 5
```

How it works:

1. Init	int i = 1 → counter starts at 1
2. Check	i <= 5 → is 1 less than or equal to 5? Yes → run body
3. Run	Print "Number: 1"
4. Update	i++ → i becomes 2, go back to step 2
5. Stop	When i = 6, condition is false → loop ends

2. The while Loop

A **while loop** keeps running as long as a condition is **true**. Use it when you don't know in advance how many times to loop.

Syntax:

```
while (condition) {  
  // code to repeat  
}
```

Example — Countdown from 5:

```
int count = 5;  
while (count > 0) {  
  System.out.println("Countdown: " + count);  
  count--;  
}  
System.out.println("Blast off!");
```

■ **Warning — Infinite Loop:** If the condition never becomes false, the loop runs forever and crashes your program. Always make sure something inside the loop moves you toward the exit condition!

3. The do-while Loop

A **do-while loop** is like a while loop, except the body runs **at least once** before checking the condition.

Syntax:

```
do {  
  // code to repeat  
} while (condition);
```

Example — Ask user for valid input:

```
int num;
do {
    System.out.print("Enter a positive number: ");
    num = scanner.nextInt();
} while (num <= 0);
// Guarantees at least one prompt is shown
```

4. The for-each Loop

A **for-each loop** (also called enhanced for) iterates over every element in an array or list without needing an index variable.

Syntax (Java):

```
for (Type item : collection) {
    // use item
}
```

Example — Print all fruits:

```
String[] fruits = {"Apple", "Banana", "Cherry"};
for (String fruit : fruits) {
    System.out.println(fruit);
}
// Output: Apple Banana Cherry
```

5. Loop Control Statements

Use these keywords to control loop execution:

Keyword	What it does	Example
break	Exits the loop immediately	Stop when you find a match
continue	Skips the rest of this iteration	Skip even numbers
return	Exits the entire function/method	Return result from inside a loop

break example:

```
for (int i = 0; i < 10; i++) {
```

```
if (i == 5) break; // stop at 5
System.out.println(i);
} // prints 0 1 2 3 4
```

continue example:

```
for (int i = 0; i < 6; i++) {
    if (i % 2 == 0) continue; // skip even
    System.out.println(i);
} // prints 1 3 5
```

6. Nested Loops

A loop inside another loop is called a **nested loop**. The inner loop runs completely for every single iteration of the outer loop.

Example — Print a 3×3 grid:

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 3; col++) {
        System.out.print(row + "," + col + " ");
    }
    System.out.println();
}
```

```
// Output:
// 1,1 1,2 1,3
// 2,1 2,2 2,3
// 3,1 3,2 3,3
```

***Tip:** Each extra level of nesting multiplies execution time. A triple-nested loop over 100 items runs 1,000,000 iterations — use with care!*

Quick Comparison

Loop	Condition check	Min runs	Best for
for	Before body	0	Known count
while	Before body	0	Unknown count
do-while	After body	1	Input validation

for-each	Auto (per item)	0	Arrays / lists
----------	-----------------	---	----------------

Practice makes perfect — write loops every day!